

参考答案

2.7

Consider the bank database of Figure 2.18. Give an expression in the relational algebra for each of the following queries.

a.

$$\Pi_{branch_name} (\sigma_{branch_city="Chicago"} (branch))$$

b.

$$\Pi_{ID} (\sigma_{branch_name="Downtown"} (borrower \bowtie_{borrower.loan_number=loan.loan_number} loan))$$

2.9

a.

$$\Pi_{ID} (\Pi_{ID,course_id}(takes) \div \Pi_{course_id} (\sigma_{dept_name="Comp. Sci"} (course)))$$

b.

The required expression is as follows:

$$r \leftarrow \Pi_{ID,course_id}(takes)$$

$$s \leftarrow \Pi_{course_id} (\sigma_{dept_name="Comp. Sci"} (course))$$

$$\Pi_{ID}(takes) - \Pi_{ID} ((\Pi_{ID}(takes) \times s) - r)$$

In general, let $r(R)$ and $s(S)$ be given, with

$$S \subseteq R$$

Then we can express the division operation using basic relational algebra operations as follows:

$$r \div s = \Pi_{R-S}(r) - \Pi_{R-S} ((\Pi_{R-S}(r) \times s) - \Pi_{R-S,S}(r))$$

To see that this expression is true, we observe that

$$\Pi_{R-S}(r)$$

gives us all tuples t that satisfy the first condition of the definition of division. The expression on the right side of the set difference operator

$$\Pi_{R-S} ((\Pi_{R-S}(r) \times s) - \Pi_{R-S,S}(r))$$

serves to eliminate those tuples that fail to satisfy the second condition of the definition of division.

Let us see how it does so. Consider

$$\Pi_{R-S}(r) \times s$$

This relation is on schema R, and pairs every tuple in

$$\Pi_{R-S}(r)$$

with every tuple in s. The expression

$$\Pi_{R-S,S}(r)$$

merely reorders the attributes of r.

Thus,

$$(\Pi_{R-S}(r) \times s) - \Pi_{R-S,S}(r)$$

gives us those pairs of tuples from

$$\Pi_{R-S}(r)$$

and s that do not appear in r. If a tuple t_j is in

$$\Pi_{R-S}((\Pi_{R-S}(r) \times s) - \Pi_{R-S,S}(r))$$

then there is some tuple t_s in s that does not combine with tuple t_j to form a tuple in r. Thus, t_j holds a value for attributes R - S that does not appear in

$$r \div s$$

It is these values that we eliminate from

$$\Pi_{R-S}(r)$$

3.4

a. Find the total number of people who owned cars that were involved in accidents in 2017.

Note: This is not the same as the total number of accidents in 2017. We must count people with several accidents only once. Furthermore, note that the question asks for owners, and it might be that the owner of the car was not the driver actually involved in the accident.

```
1 select count (distinct person.driver_id)
2 from accident, participated, person, owns
3 where accident.report_number = participated.report_number
4       and owns.driver_id = person.driver_id
5       and owns.license_plate = participated.license_plate
6       and year = 2017
```

b. Delete all year-2010 cars belonging to the person whose ID is '12345'.

```
1 delete car
2 where year = 2010
3    and license_plate in (
4        select license_plate
5        from owns o
6        where o.driver_id = '12345'
7    )
```

Note: The `owns`, `accident` and `participated` records associated with the deleted cars still exist.

3.9

Employee database:

```
1 employee (ID, person_name, street, city)
2 works (ID, company_name, salary)
3 company (company_name, city)
4 manages (ID, manager_id)
```

a. Find the ID, name, and city of residence of each employee who works for "First Bank Corporation".

```
1 select e.ID, e.person_name, city
2 from employee as e, works as w
3 where w.company_name = 'First Bank Corporation'
4    and w.ID = e.ID
```

b. Find the ID, name, and city of residence of each employee who works for "First Bank Corporation" and earns more than \$10000.

```
1 select *
2 from employee
3 where ID in (
4     select ID
5     from works
6     where company_name = 'First Bank Corporation'
7         and salary > 10000
8 )
```

This could be written also in the style of the answer to part a.

c. Find the ID of each employee who does not work for "First Bank Corporation".

```
1 select ID
2 from works
3 where company_name <> 'First Bank Corporation'
```

If one allows people to appear in `employee` without appearing also in `works`, the solution is slightly more complicated. An outer join as discussed in Chapter 4 could be used as well.

```
1 select ID
2 from employee
3 where ID not in (
4     select ID
5     from works
6     where company_name = 'First Bank Corporation'
7 )
```

d. Find the ID of each employee who earns more than every employee of "Small Bank Corporation".

```
1 select ID
2 from works
3 where salary > all (
4     select salary
5     from works
6     where company_name = 'Small Bank Corporation'
7 )
```

If people may work for several companies and we wish to consider the total earnings of each person, the problem is more complex. But note that the fact that `ID` is the primary key for `works` implies that this cannot be the case.

e. Assume that companies may be located in several cities. Find the name of each company that is located in every city in which "Small Bank Corporation" is located.

```
1 select S.company_name
2 from company as S
3 where not exists (
4     (
5         select city
6         from company
7         where company_name = 'Small Bank Corporation'
8     )
9     except
10    (
11        select city
12        from company as T
13        where S.company_name = T.company_name
```

```
14 | )  
15 | )
```

f. Find the name of the company that has the most employees, or companies in the case where there is a tie for the most.

```
1 | select company_name  
2 | from works  
3 | group by company_name  
4 | having count (distinct ID) >= all (  
5 |     select count (distinct ID)  
6 |     from works  
7 |     group by company_name  
8 | )
```

g. Find the name of each company whose employees earn a higher salary, on average, than the average salary at "First Bank Corporation".

```
1 | select company_name  
2 | from works  
3 | group by company_name  
4 | having avg (salary) > (  
5 |     select avg (salary)  
6 |     from works  
7 |     where company_name = 'First Bank Corporation'  
8 | )
```

4.2

a. Display a list of all instructors, showing each instructor's ID and the number of sections taught.

Make sure to show the number of sections as 0 for instructors who have not taught any section. The query should use an outer join and should not use subqueries.

```
1 | select ID, count(sec_id) as Number_of_sections  
2 | from instructor natural left outer join teaches  
3 | group by ID
```

The above query should not be written using `count(*)` since that would count null values also. It could be written using any attribute from `teaches` which does not occur in `instructor`, which would be correct although it may be confusing to the reader. Attributes that occur in `instructor` would not be null even if the instructor has not taught any section.

b. Write the same query as above, but using a scalar subquery, and not using outer join.

```
1 select ID,  
2       (select count(*) as Number_of_sections  
3         from teaches T  
4         where T.id = I.id)  
5 from instructor I
```

c. Display the list of all course sections offered in Spring 2018, along with the ID and name of each instructor teaching the section.

If a section has more than one instructor, that section should appear as many times in the result as it has instructors. If a section does not have any instructor, it should still appear in the result with the instructor name set to "-".

```
1 select course_id, sec_id, ID,  
2       decode(name, null, '-', name) as name  
3 from (section natural left outer join teaches)  
4       natural left outer join instructor  
5 where semester = 'Spring'  
6       and year = 2018
```

The query may also be written using the `coalesce` operator, by replacing `decode(..)` with `coalesce(name, '-')`. A more complex version of the query can be written using union of join result with another query that uses a subquery to find courses that do not match.

d. Display the list of all departments, with the total number of instructors in each department, without using subqueries.

Make sure to show departments that have no instructors, and list those departments with an instructor count of zero.

```
1 select dept_name, count(ID)  
2 from department natural left outer join instructor  
3 group by dept_name
```

4.5

a.

Consider the case where a professor in the Physics department teaches an Elec. Eng. course. Even though there is a valid corresponding entry in `teaches`, it is lost in the natural join of `instructor`, `teaches`, and `course`, since the instructor's department name does not match the department name of the course.

A dataset corresponding to the same is:

```

1 | instructor = {('12345', 'Gauss', 'Physics', 10000)}
2 | teaches = {('12345', 'EE321', 1, 'Spring', 2017)}
3 | course = {('EE321', 'Magnetism', 'Elec. Eng.', 6)}

```

b.

The query in question 4.2(a) is a good example for this. Instructors who have not taught a single course should have number of sections as 0 in the query result.

Many other similar examples are possible.

c.

Consider the query:

```

1 | select *
2 | from teaches natural join instructor;

```

In this query, we would lose some sections if `teaches.ID` is allowed to be null and such tuples exist. If, just because `teaches.ID` is a foreign key to `instructor`, we did not create such a tuple, the error in the above query would not be detected.

5.7

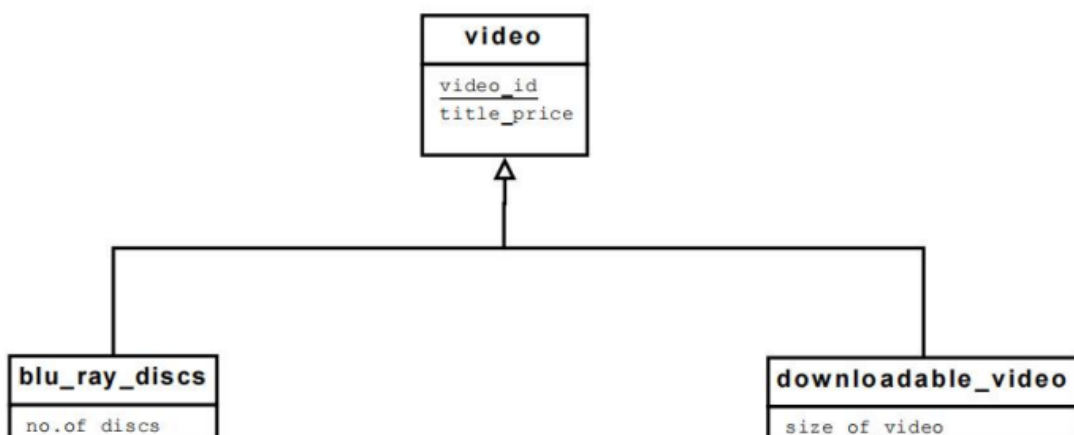
```

1 | create trigger check-delete-trigger after delete on account
2 | referencing old row as orow
3 | for each row
4 | delete from depositor
5 | where depositor.customer_name not in
6 |   (select customer_name
7 |    from depositor
8 |    where account_number <> orow.account_number)
9 | end

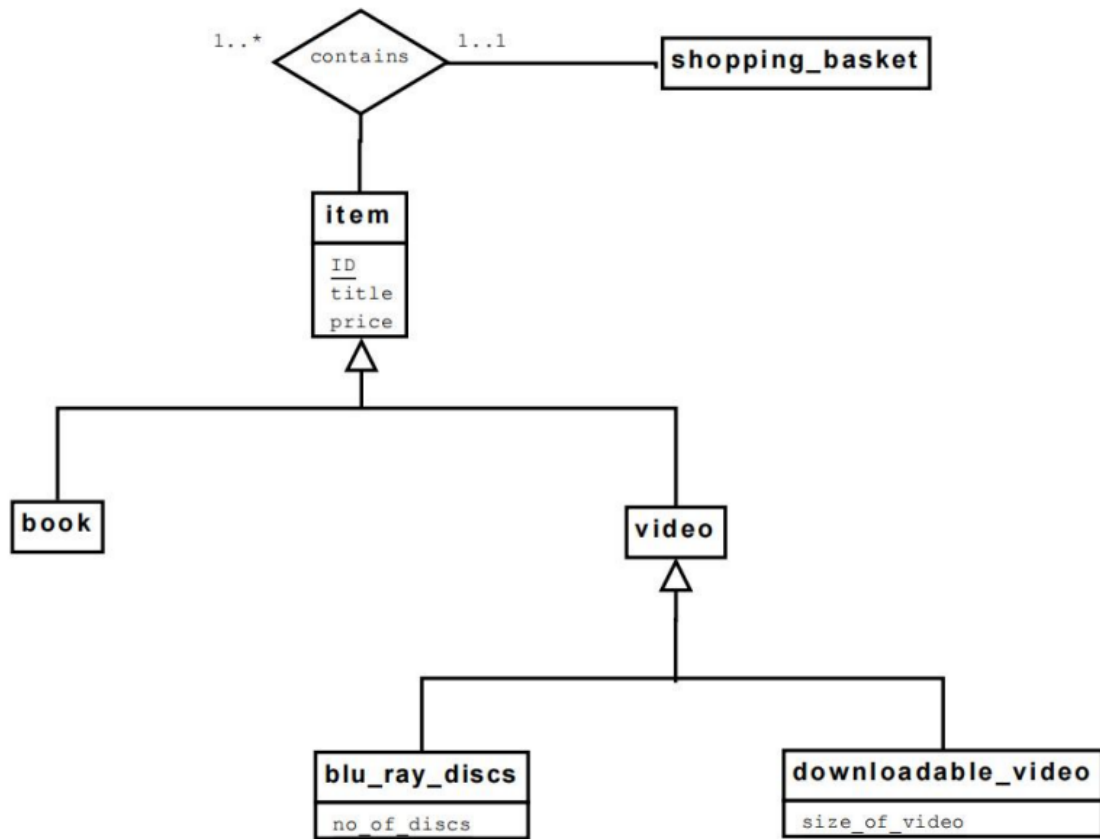
```

6.21

a.

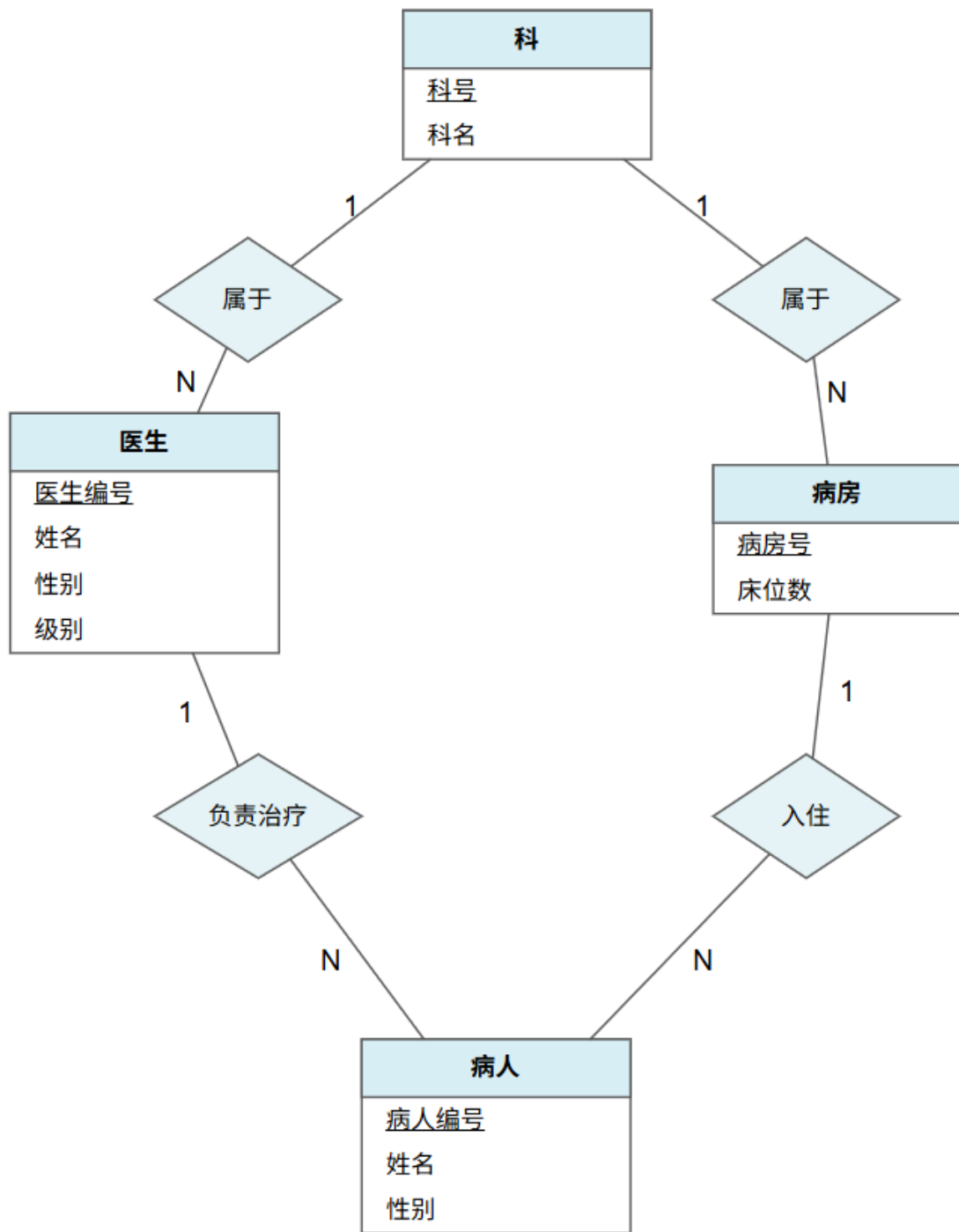


b.



6.题目1

ER图



关系模式集合

科(科号, 科名)

主码：科号

医生(医生编号, 姓名, 性别, 级别, 科号)

主码：医生编号

外码：科号 → 科(科号)

病房(病房号, 床位数, 科号)

主码：病房号

外码：科号 → 科(科号)

病人(病人编号, 姓名, 性别, 病房号, 医生编号)

主码: 病人编号

外码: 病房号 $\rightarrow$ 病房(病房号)

外码: 医生编号 $\rightarrow$ 医生(医生编号)

7.30

a.

We compute the closure of (B^+) :

1	$B \rightarrow BD$	(third dependency)
2	$BD \rightarrow ABD$	(fourth dependency)
3	$ABD \rightarrow ABCD$	(first dependency)
4	$ABCD \rightarrow ABCDE$	(second dependency)

Thus,

1	$B^+ = ABCDE$
---	---------------

b.

Prove (using Armstrong's axioms) that (AF) is a superkey.

1	$A \rightarrow BCD$	(Given)
2	$A \rightarrow ABCD$	(Augmentation with A)
3		
4	$BC \rightarrow DE$	(Given)
5	$ABCD \rightarrow ABCDE$	(Augmentation with ABCD)
6		
7	$A \rightarrow ABCDE$	(Transitivity)
8		
9	$AF \rightarrow ABCDEF$	(Augmentation with F)

Hence, (AF) is a superkey.

c.

We see that (D) is extraneous in dependency 1 and 2 because of dependency 3.

Removing these two occurrences gives the new set of dependencies:

1	$A \rightarrow BC$
2	$BC \rightarrow E$
3	$B \rightarrow D$
4	$D \rightarrow A$

Now notice that:

```
1 | B+ = ABCDE
```

In particular, the FD ($B \rightarrow E$) can be derived from this set.

Thus, the attribute (C) is extraneous in the third dependency.

Removing (C), and combining with the third FD, we get the final canonical cover:

```
1 | A → BC
2 | B → DE
3 | D → A
```

Here, no attribute is extraneous in any FD.

d.

There is no FD in the canonical cover such that its set of attributes is a subset of another FD in the canonical cover.

Thus, each FD gives rise to its own relation:

```
1 | r1(A, B, C)
2 | r2(B, D, E)
3 | r3(D, A)
```

Now notice that attribute (F) is not dependent on any attribute.

Thus, (F) must be part of every superkey.

Since none of the above relations contains (F), none of them contains a superkey.

Therefore, we add a new relation containing a superkey:

```
1 | r4(A, F)
```

e.

We start with:

```
1 | r(A, B, C, D, E, F)
```

The relation is not in BCNF because of the first FD.

Decompose accordingly:

```
1 | r1(A, B, C, D)
2 | r2(A, E, F)
```

Now notice that:

```
1 | A → E
```

is an FD in (F^+) , and it causes (r_2) to violate BCNF.

Decomposing (r_2) again gives:

- | | |
|---|-------------------|
| 1 | $r_1(A, B, C, D)$ |
| 2 | $r_2(A, F)$ |
| 3 | $r_3(A, E)$ |

This schema is now in BCNF.